

Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации

*Ордена Трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования*

**«Московский технический университет связи и информатики»
(МТУСИ)**

Лабораторная работа № 4

по дисциплине “Информационные технологии и программирование”

Выполнила:

студентка группы БВТ2301

Чиняева Алиса Юрьевна

Проверил:

Харрасов К.Р.

Лабораторная работа № 4

Исключения в Java являются механизмом обработки ошибок и нестандартных ситуаций во время выполнения программы. Они позволяют изолировать ошибки и обрабатывать их в специальном блоке кода, что делает программу более надежной и устойчивой к ошибкам.

В Java есть множество типов исключений, которые могут возникнуть в различных ситуациях. Например, классы `IOException` и `FileNotFoundException` используются для обработки ошибок ввода/вывода, а классы `NullPointerException` и `ArrayIndexOutOfBoundsException` используются для обработки ошибок, связанных с работой с объектами и массивами.

Кроме того, в Java можно создавать свои собственные исключения, наследуясь от класса `Exception` или его подклассов. Это позволяет создавать более гибкую и точечную обработку ошибок в своих программах.

При работе с исключениями важно учитывать, что блок `try-catch` не должен заменять проверку условий и предотвращение возможных ошибок на этапе написания кода. Он должен использоваться только для обработки неожиданных ошибок, которые могут возникнуть в процессе выполнения программы. Код, который может вызвать исключение, помещается в блок `try`. Если исключение возникает внутри блока `try`, то управление передается в блок `catch`, который обрабатывает исключение. Блок `catch` может содержать несколько разделов, каждый из которых обрабатывает определенный тип исключения.

Задание 1:

Необходимо написать программу, которая будет находить среднее арифметическое элементов массива. При этом программа должна обрабатывать ошибки, связанные с выходом за границы массива и неверными данными (например, если элемент массива не является числом).

Для начала необходимо создать массив, в котором можно будет хранить и числа и строки, для этого воспользуемся классом `Object[]`. Также создаем 2 переменные для подсчета суммы и чисел в нашем массиве.

Конструкция `try-catch` будет просматривать цикл `for`, в котором также будет идти проверка на соответствие элемента числу. Если возникло несоответствие, то выкидывается исключение, если же нет, то число складывается с `sum` и также `count` прибавляет 1.

```

public class ArrayAverage {
    Run | Debug
    public static void main(String[] args) {
        Object[] arr = {1, 2, "abc", 4, 5};
        int sum = 0;
        int count = 0;
        try {
            for (int i = 0; i < arr.length; i++) {
                if (arr[i] instanceof Integer) {
                    sum += (Integer) arr[i];
                    count++;
                } else {
                    throw new IllegalArgumentException("неверно введены данные");
                }
            }
            double average = (double) sum / count;
            System.out.println("Среднее арифметическое = " + average);
        }
    }
}

```

В конце, если программа завершилась без исключений досчитывается среднее арифметическое.

Если же вышло исключение, то мы их ловим

```

        } catch (IllegalArgumentException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
        } catch (ArrayIndexOutOfBoundsException e) {
            //выход за границы массива
            System.out.println("Ошибка: " + e.getMessage());
        }
    }
}

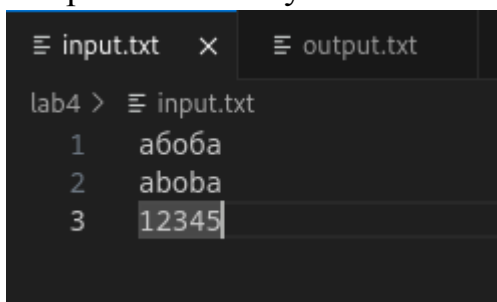
```

Задание 2:

Необходимо написать программу, которая будет копировать содержимое одного файла в другой. При этом программа должна обрабатывать возможные ошибки, связанные с:

Вариант 1	Вариант 2
Открытием и закрытием файлов	Чтение и записью файлов

Создадим файлы, из которого будем копировать и который будем копировать, в моем случае это input.txt и output.txt. В input мы запишем некоторую информацию, когда output оставим пустым.



```

lab4 > input.txt
1  абоба
2  aboba
3  12345

```

Для работы с файлами, чтением и записью нам необходимы следующие библиотеки и **IOException** для обработки ошибок ввода-вывода.

```
import java.io.FileInputStream;  
import java.io.FileOutputStream;  
import java.io.IOException;
```

Переменные **input** и **output** содержат названия исходного и целевого файлов. Используется конструкция **try-with-resources**, которая автоматически закрывает файлы после окончания работы с ними.

- **FileInputStream fin = new FileInputStream(input)** — открывает файл **input.txt** для чтения.
- **FileOutputStream fout = new FileOutputStream(output)** — открывает файл **output.txt** для записи.

Объявляется буфер **temp** размером 1024 байта для временного хранения прочитанных данных. Переменная **bytesRead** будет хранить количество байт, считанных за одну операцию.

Сообщение о завершении копирования выводится на экран, если копирование прошло успешно.

```
public class FileCopy {  
    Run | Debug  
    public static void main(String[] args) {  
        String input = "input.txt";  
        String output = "output.txt";  
  
        try (FileInputStream fin = new FileInputStream(input);  
            FileOutputStream fout = new FileOutputStream(output)) {  
  
            byte[] temp = new byte[1024];  
            int bytesRead;  
  
            while ((bytesRead = fin.read(temp)) != -1) {  
                fout.write(temp, 0, bytesRead);  
            }  
            System.out.println("Копирование завершено.");  
        } catch (IOException e) {  
            System.out.println("Ошибка: " + e.getMessage());  
        }  
    }  
}
```

catch блок перехватывает исключения **IOException**, которые могут возникнуть при работе с файлами (если файл не найден или произошла ошибка чтения/записи).

Задание 3:

Вариант 2: Создайте класс CustomAgeException, который будет использоваться для обработки недопустимых возрастов. Реализуйте программу, которая проверяет возраст пользователя с использованием этого класса, и, если возраст меньше 0 или больше 120, выбрасывайте исключение CustomAgeException.

Прописываем класс CustomAgeException, это класс-исключение, поэтому наследуем для него класс Exception. Прописываем сообщение, которое будем передавать

```
package lab4;

public class GateAge {
    public static void checkAge(int age) throws CustomAgeException {
        if (age < 0) {
            throw new CustomAgeException(message:"слишком маленький");
        } else if (age > 120) {
            throw new CustomAgeException(message:"слишком большой");
        }
        System.out.println("Подходит :)");
    }

    Run | Debug
    public static void main(String[] args) {
        int userAge = 125;

        try {
            checkAge(userAge);
        } catch (CustomAgeException e) {
            System.out.println("Error: " + e.getMessage());
            ExceptionLogger.log(e);
        }
    }
}
```

```
lab4 > CustomAgeException.java > ...
1 package lab4;
2
3 public class CustomAgeException extends Exception {
4     public CustomAgeException(String message) {
5         super(message);
6     }
7
8     public CustomAgeException() {
9         super();
10    }
11 }
12
13
14
```

```
package lab4;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class ExceptionLogger {
    private static final String LOG_FILE = "exception.log";

    public static void log (Exception e) {
        try (BufferedWriter buffer = new BufferedWriter(new FileWriter(LOG_FILE, true))) {
            buffer.write(e.toString());
            buffer.newLine();
        }
        catch (IOException er) {
            System.out.println("Error while logging");
        }
    }
}
```